OpenCV Lab Assessment – Five Group Versions

Faculty of Artificial Intelligence & Robotics — Computer Vision Course

Assessment Type: Practical Lab Quiz (5 Equivalent Versions)	**Target Audience:** Computer Vision & Robotics Students
Total Time Allowed: 40 Minutes	**Total Marks:** 20 Marks (MCQ: 14 Marks | Written: 6 Marks)
Curriculum Scope: Lab 1 & Lab 2 (Core Fundamentals & Geometry)	**Fairness Guarantee:** 100% Equivalent Cognitive Difficulty Matrix

Assessment Blueprint & Strict Equivalence Matrix Across Groups

Q#	Cognitive Level	Curriculum Topic	Target Skill & Learning Outcome	Marks
Q1	Medium MCQ	Image Representation & Memory	NumPy shape (H, W, C), uint8 type, and uncompressed byte size calculation	2
Q2	Medium MCQ	Color Spaces & Channel Properties	BGR vs RGB display, HSV ranges, Grayscale luminance weights, CIELAB	2
Q3	Medium MCQ	Resizing & Interpolation Algorithms	Interpolation flag selection (AREA, CUBIC, NEAREST, LINEAR, LANCZOS4)	2
Q4	Medium MCQ	Geometric Flipping & Rotations	Flip codes (0: vertical, 1: horizontal, -1: both) and fixed 90° rotation codes	2
Q5	Challenging MCQ	ROI Slicing: Memory View vs Copy	Code execution: Understanding shared memory buffer mutation vs .copy()	2
Q6	Challenging MCQ	Image Arithmetic: Saturation vs Modulo	Code execution: OpenCV saturation clamping (255/0) vs NumPy 8-bit wrap (% 256)	2
Q7	Challenging MCQ	Affine Matrix & Coordinate Inversion	Coordinate rules: Center (x, y) vs shape (H, W) and warpAffine dsize (W, H)	2
Q8	Written Scenario	Practical Problem Solving & Function Selection	Identifying OpenCV functions (Border padding, Equalization, Blending, Masking, Homography)	6

Instructions for Examination Administration & Students:

1. **Version Integrity:** Students must only solve the assessment version assigned to their registered group (Group One to Group Five).
2. **Single-Page Assessment:** Each group's assessment is strictly self-contained on its designated single page. Do not detach pages.
3. **Multiple Choice Questions (Q1–Q7):** Exactly one option (A, B, C, or D) is correct per question. Clearly circle or write your chosen letter.
4. **Written Scenario Question (Q8):** Answer clearly in the designated spaces. State the exact OpenCV function name, required arguments, and a concise technical justification.
5. **Instructor Reference:** A comprehensive answer key and grading rubric is appended at the end of this document (Instructor Copy only).

Examiner / Instructor: _____ **Head of Department:** _____

OPENCV PRACTICAL ASSESSMENT — GROUP ONE

Total Time: 40 Mins | Total Marks: 20

Course: Computer Vision & Robotics | Coverage: Lab 1 & Lab 2

Marks: Q1–Q7 (2 pts each) | Q8 (6 pts)

Student Name: _____ **Student ID:** _____ **Date:** _____

Q1. **[Medium MCQ]** A 3-channel color image is loaded with cv2.imread('sample.jpg'). Running img.shape returns (400, 600, 3) with data type uint8. What are the image height, image width, and the exact uncompressed memory size occupied by the image array in bytes?

A) Height = 600, Width = 400, Memory = 240,000 bytes
B) Height = 400, Width = 600, Memory = 720,000 bytes
C) Height = 400, Width = 600, Memory = 240,000 bytes
D) Height = 600, Width = 400, Memory = 720,000 bytes

Q2. **[Medium MCQ]** When a color photo loaded with cv2.imread() is displayed using matplotlib.pyplot.imshow(), a red vehicle appears bright blue. Which OpenCV statement correctly converts the image buffer so that Matplotlib displays true realistic colors?

A) img = cv2.cvtColor(img, cv2.COLOR_RGB2BGR)
B) img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
C) img = cv2.split(img)[::-1]
D) img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

Q3. **[Medium MCQ]** An engineer needs to downscale a 4K camera frame (3840×2160) to standard resolution (640×360). According to OpenCV design principles, which interpolation method is specifically recommended to prevent moiré patterns and aliasing artifacts when shrinking images?

A) cv2.INTER_AREA
B) cv2.INTER_NEAREST
C) cv2.INTER_CUBIC
D) cv2.INTER_LINEAR

Q4. **[Medium MCQ]** A robotic camera mounted upside-down captures inverted images. The vision engineer executes frame = cv2.flip(frame, 0). Which geometric transformation does this operation perform on the image?

A) Horizontal mirror reflection across the vertical y-axis
B) 90-degree clockwise rotation
C) Vertical flip across the horizontal x-axis (upside down)
D) Simultaneous horizontal and vertical flip (180-degree rotation)

Q5. **[Challenging MCQ]** Analyze the following Python snippet:
img = np.zeros((200, 200, 3), dtype=np.uint8); roi = img[50:150, 50:150]
roi[:] = [0, 255, 0]; print(img[100, 100].tolist())
What is the printed output and the underlying memory reason?

A) [0, 255, 0] — Slicing creates a memory view, directly modifying the original image buffer.
B) [0, 0, 0] — NumPy slices create independent deep copies, keeping img unchanged.
C) [255, 255, 255] — The assignment broadcasts white pixels across overlapping channels.
D) IndexError — Coordinate (100, 100) falls outside the bounded region of interest.

Q6. **[Challenging MCQ]** Consider two single-pixel 8-bit unsigned integer arrays evaluated in Python:
p1 = np.array([210], dtype=np.uint8); p2 = np.array([70], dtype=np.uint8)
out1 = cv2.add(p1, p2)[0][0]; out2 = (p1 + p2)[0]
What are the values of out1 and out2?

A) out1 = 280, out2 = 280
B) out1 = 255, out2 = 24
C) out1 = 24, out2 = 255
D) out1 = 255, out2 = 255

Q7. **[Challenging MCQ]** An image img has shape (400, 600, 3) (height=400, width=600). A student wants to rotate it 45° counter-clockwise around its geometric center at 1.0x scale. Which function calls correctly create the matrix and warp the image?

A) M = cv2.getRotationMatrix2D((200, 300), 45, 1.0); res = cv2.warpAffine(img, M, (400, 600))
B) M = cv2.getRotationMatrix2D((300, 200), -45, 1.0); res = cv2.warpAffine(img, M, (400, 600))
C) M = cv2.getRotationMatrix2D((200, 300), 45, 1.0); res = cv2.warpAffine(img, M, (600, 400))
D) M = cv2.getRotationMatrix2D((300, 200), 45, 1.0); res = cv2.warpAffine(img, M, (600, 400))

Q8. [Written Scenario-Based Problem — 6 Marks] Topic: Aspect Ratio Preservation via Border Padding

Problem Scenario: An autonomous mobile robot captures rectangular images (640×480) that must be fed into a deep neural network expecting a fixed square 640×640 input. Resizing directly squashes objects and distorts aspect ratios, while cropping removes peripheral obstacle data. Instead, the perception engineer keeps the aspect ratio and needs to add 80 pixels of solid black padding to both top and bottom borders without altering the original content.

(a) Most Appropriate OpenCV Function: _____

(b) Essential Function Call / Parameters: _____

(c) Technical Justification & Suitability:

_____

_____

OPENCV PRACTICAL ASSESSMENT — GROUP TWO

Total Time: 40 Mins | Total Marks: 20

Course: Computer Vision & Robotics | Coverage: Lab 1 & Lab 2

Marks: Q1–Q7 (2 pts each) | Q8 (6 pts)

Student Name: _____ **Student ID:** _____ **Date:** _____

Q1. [Medium MCQ] A single-channel grayscale image is read using img = cv2.imread('scan.png', cv2.IMREAD_GRAYSCALE). Running img.shape returns (500, 800) with data type uint8. What are the image height, image width, and the exact uncompressed memory footprint of this array in bytes?

A) Height = 800, Width = 500, Memory = 400,000 bytes
B) Height = 500, Width = 800, Memory = 1,200,000 bytes
C) Height = 500, Width = 800, Memory = 400,000 bytes
D) Height = 800, Width = 500, Memory = 1,200,000 bytes

Q2. [Medium MCQ] An image is converted to the HSV color space using hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV). What does the Hue (H) channel represent, and what is its standard numerical integer range for standard 8-bit unsigned images (uint8) in OpenCV?

A) Dominant color wavelength / tint, represented in the range 0 to 179
B) Color purity / saturation level, represented in the range 0 to 255
C) Perceptual luminance / brightness, represented in the range 0 to 360
D) Dominant color wavelength / tint, represented in the range 0 to 360

Q3. [Medium MCQ] A student needs to enlarge a low-resolution thumbnail (32×32) to 256×256 for a robotics dashboard. High visual smoothness and smooth gradients are prioritized over processing time. Which interpolation method computes new pixels using a 4×4 neighborhood bicubic spline?

A) cv2.INTER_NEAREST
B) cv2.INTER_CUBIC
C) cv2.INTER_AREA
D) cv2.INTER_LINEAR

Q4. [Medium MCQ] An engineer working on autonomous driving data augmentation needs to horizontally mirror road images across the vertical y-axis (flipping left and right). Which OpenCV call achieves this transformation?

A) cv2.flip(img, 0)
B) cv2.flip(img, -1)
C) cv2.rotate(img, cv2.ROTATE_180)
D) cv2.flip(img, 1)

Q5. [Challenging MCQ] Analyze the following Python snippet:

```
img = np.zeros((200, 200, 3), dtype=np.uint8); img[100, 100] = [120, 120, 120]
roi = img[50:150, 50:150].copy(); roi[:] = [255, 0, 0]
print(img[100, 100].tolist())
```

What is the printed output and why?

A) [255, 0, 0] — The ROI slice remains bound to the original array buffer.
B) [0, 0, 0] — Reassigning the ROI resets surrounding pixels in the source array.
C) [120, 120, 120] — Calling .copy() created an independent buffer, leaving img unchanged.
D) [255, 255, 255] — The assignment cascades across all channels.

Q6. [Challenging MCQ] Consider two single-pixel 8-bit unsigned integer arrays evaluated in Python:

```
p1 = np.array([40], dtype=np.uint8); p2 = np.array([90], dtype=np.uint8)
out1 = cv2.subtract(p1, p2)[0][0]; out2 = (p1 - p2)[0]
```

What are the values of out1 and out2?

A) out1 = 0, out2 = 206
B) out1 = 206, out2 = 0
C) out1 = -50, out2 = -50
D) out1 = 0, out2 = 0

Q7. [Challenging MCQ] An image img has shape (300, 500, 3) (height=300, width=500). A student wants to rotate the image 90° counter-clockwise around its center and scale it up by 1.5×. Which parameters correctly invoke getRotationMatrix2D and warpAffine?

A) M = cv2.getRotationMatrix2D((150, 250), 90, 1.5); res = cv2.warpAffine(img, M, (300, 500))
B) M = cv2.getRotationMatrix2D((250, 150), 90, 1.5); res = cv2.warpAffine(img, M, (500, 300))
C) M = cv2.getRotationMatrix2D((250, 150), -90, 1.5); res = cv2.warpAffine(img, M, (500, 300))
D) M = cv2.getRotationMatrix2D((150, 250), 90, 1.0); res = cv2.warpAffine(img, M, (500, 300))

Q8. [Written Scenario-Based Problem — 6 Marks] Topic: Contrast Enhancement via Histogram Equalization

Problem Scenario: An autonomous delivery rover operating at dusk captures grayscale navigation camera feeds. Due to heavy fog and twilight, the images suffer from severe low contrast; nearly all pixel intensities are crammed into a narrow dark band [25, 75]. Visual features such as curbstones and obstacles cannot be reliably detected. The vision system needs to automatically redistribute the intensity distribution across the full dynamic range [0, 255] on the grayscale feed.

(a) Most Appropriate OpenCV Function: _____

(b) Essential Function Call / Parameters: _____

(c) Technical Justification & Suitability:

_____

_____

OPENCV PRACTICAL ASSESSMENT — GROUP THREE

Total Time: 40 Mins | Total Marks: 20

Course: Computer Vision & Robotics | Coverage: Lab 1 & Lab 2

Marks: Q1–Q7 (2 pts each) | Q8 (6 pts)

Student Name: _____ **Student ID:** _____ **Date:** _____

Q1. [Medium MCQ] A color image is loaded with img = cv2.imread('robot.jpg'). Running img.shape returns (300, 700, 3) with data type uint8. What are the image height, image width, and the total uncompressed memory size occupied by the image array in bytes?

A) Height = 300, Width = 700, Memory = 630,000 bytes
B) Height = 700, Width = 300, Memory = 630,000 bytes
C) Height = 300, Width = 700, Memory = 210,000 bytes
D) Height = 700, Width = 300, Memory = 210,000 bytes

Q2. [Medium MCQ] When converting a 3-channel BGR image to single-channel grayscale with cv2.cvtColor(img, cv2.COLOR_BGR2GRAY), how does OpenCV compute each output grayscale intensity Y from the Blue, Green, and Red channels?

A) Simple arithmetic mean: Y = (B + G + R) / 3
B) Median selection: Y = median(B, G, R)
C) Weighted perceptual luminance sum: Y = 0.299*R + 0.587*G + 0.114*B
D) Maximum channel intensity selection: Y = max(B, G, R)

Q3. [Medium MCQ] In an ultra-high-speed robotic tracking system running on an embedded microcontroller, frames must be downscaled as fast as possible. Visual smoothness is secondary to minimal processing latency. Which interpolation method provides the fastest execution speed?

A) cv2.INTER_LANCZOS4
B) cv2.INTER_CUBIC
C) cv2.INTER_LINEAR
D) cv2.INTER_NEAREST

Q4. [Medium MCQ] A miniature camera mounted on an underwater vehicle transmits video frames that are inverted both vertically and horizontally. Which single OpenCV function call rectifies the image along both axes simultaneously?

A) cv2.flip(frame, 0)
B) cv2.flip(frame, -1)
C) cv2.flip(frame, 1)
D) cv2.rotate(frame, cv2.ROTATE_90_CLOCKWISE)

Q5. [Challenging MCQ] Analyze the following Python snippet:
img = np.zeros((150, 150, 3), dtype=np.uint8); patch = img[30:100, 40:110]
patch[:, :] = [0, 0, 255]; print(img[50, 60].tolist())
What will be printed to the console?

A) [0, 0, 255]
B) [0, 0, 0]
C) [255, 0, 0]
D) IndexError: array dimension mismatch

Q6. [Challenging MCQ] Consider two single-pixel 8-bit unsigned integer arrays evaluated in Python:
p1 = np.array([190], dtype=np.uint8); p2 = np.array([80], dtype=np.uint8)
out1 = cv2.add(p1, p2)[0][0]; out2 = (p1 + p2)[0]
What are the values of out1 and out2?

A) out1 = 270, out2 = 270
B) out1 = 14, out2 = 255
C) out1 = 255, out2 = 14
D) out1 = 255, out2 = 255

Q7. [Challenging MCQ] An image img has dimensions (height=400, width=600). A student wants to translate the image by shifting it 60 pixels to the right (+x) and 40 pixels downwards (+y). What is the correct translation matrix M and call to cv2.warpAffine?

A) M = np.float32([[1, 0, 40], [0, 1, 60]]); res = cv2.warpAffine(img, M, (400, 600))
B) M = np.float32([[1, 0, 60], [0, 1, 40]]); res = cv2.warpAffine(img, M, (600, 400))
C) M = np.float32([[1, 0, 60], [0, 1, 40]]); res = cv2.warpAffine(img, M, (400, 600))
D) M = np.float32([[0, 1, 60], [1, 0, 40]]); res = cv2.warpAffine(img, M, (600, 400))

Q8. [Written Scenario-Based Problem — 6 Marks] Topic: Multi-Image Blending via Linear Alpha Composition

Problem Scenario: A security surveillance control room receives two calibrated camera feeds from adjacent angles overlooking a secure facility entrance (feedA and feedB). The monitoring software needs to render a single composite overlay feed that smoothly blends both video streams, giving feedA 65% opacity/weight and feedB 35% opacity/weight, with zero additive brightness offset. Simple array addition causes severe pixel brightness clipping and color distortion.

(a) Most Appropriate OpenCV Function: _____

(b) Essential Function Call / Parameters: _____

(c) Technical Justification & Suitability:

_____

_____

OPENCV PRACTICAL ASSESSMENT — GROUP FOUR

Total Time: 40 Mins | Total Marks: 20

Course: Computer Vision & Robotics | Coverage: Lab 1 & Lab 2

Marks: Q1–Q7 (2 pts each) | Q8 (6 pts)

Student Name: _____ **Student ID:** _____ **Date:** _____

Q1. **[Medium MCQ]** An inspection camera generates high-definition color frames with shape (600, 900, 3) and data type uint8. What are the image height, image width, and the total uncompressed memory size occupied by the image array in bytes?

A) Height = 900, Width = 600, Memory = 540,000 bytes
B) Height = 600, Width = 900, Memory = 540,000 bytes
C) Height = 900, Width = 600, Memory = 1,620,000 bytes
D) Height = 600, Width = 900, Memory = 1,620,000 bytes

Q2. **[Medium MCQ]** A student separates the channels of a BGR image using b, g, r = cv2.split(img) and visualizes the blue channel with plt.imshow(b, cmap='gray'). What do the bright (near 255) pixels in this single-channel grayscale visualization represent?

A) Pixels that were pure white in the original input image only
B) Pixels that had a high-intensity blue color component in the original image
C) Pixels where red and green were dominant over blue
D) Noise artifacts introduced during the channel splitting routine

Q3. **[Medium MCQ]** When resizing an image with cv2.resize(img, (width, height)), if no interpolation argument is explicitly specified by the programmer, which interpolation algorithm does OpenCV execute by default?

A) cv2.INTER_LINEAR
B) cv2.INTER_NEAREST
C) cv2.INTER_CUBIC
D) cv2.INTER_AREA

Q4. **[Medium MCQ]** A document scanning mobile application needs to rotate an image exactly 90 degrees counter-clockwise to correct user orientation. Which OpenCV call performs this exact right-angle geometric rotation?

A) cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)
B) cv2.flip(img, -1)
C) cv2.rotate(img, cv2.ROTATE_90_COUNTERCLOCKWISE)
D) cv2.rotate(img, cv2.ROTATE_180)

Q5. **[Challenging MCQ]** Analyze the following Python snippet:

```
img = np.zeros((150, 150, 3), dtype=np.uint8); img[75, 75] = [200, 200, 200]
crop = img[50:100, 50:100].copy(); crop[:] = [0, 0, 0]
print(img[75, 75].tolist())
```

What will be printed to the console?

A) [0, 0, 0]
B) [255, 255, 255]
C) [100, 100, 100]
D) [200, 200, 200]

Q6. **[Challenging MCQ]** Consider two single-pixel 8-bit unsigned integer arrays evaluated in Python:

```
p1 = np.array([30], dtype=np.uint8); p2 = np.array([100], dtype=np.uint8)
out1 = cv2.subtract(p1, p2)[0][0]; out2 = (p1 - p2)[0]
```

What are the values of out1 and out2?

A) out1 = 186, out2 = 0
B) out1 = 0, out2 = 186
C) out1 = -70, out2 = -70
D) out1 = 0, out2 = 0

Q7. **[Challenging MCQ]** An image img has dimensions (height=500, width=700). A student wants to translate the image by shifting it 50 pixels to the left (-x direction) and 80 pixels downwards (+y direction). What is the correct matrix M and call to cv2.warpAffine?

A) M = np.float32([[1, 0, -50], [0, 1, 80]]); res = cv2.warpAffine(img, M, (700, 500))
B) M = np.float32([[1, 0, 80], [0, 1, -50]]); res = cv2.warpAffine(img, M, (700, 500))
C) M = np.float32([[1, 0, -50], [0, 1, 80]]); res = cv2.warpAffine(img, M, (500, 700))
D) M = np.float32([[0, 1, -50], [1, 0, 80]]); res = cv2.warpAffine(img, M, (500, 700))

Q8. **[Written Scenario-Based Problem — 6 Marks]** **Topic: Region Extraction via Bitwise Masking**

Problem Scenario: An autonomous robotic mower uses an AI perception model that segments turf from non-grass obstacles. The model produces a single-channel binary mask (mask) where grass pixels have value 255 (white) and obstacle/sidewalk pixels have value 0 (black). The path planner needs to extract the true color pixels of the grass from the original image (img) while setting all non-grass pixels to pure black.

(a) Most Appropriate OpenCV Function: _____

(b) Essential Function Call / Parameters: _____

(c) Technical Justification & Suitability:

_____

_____

OPENCV PRACTICAL ASSESSMENT — GROUP FIVE

Total Time: 40 Mins | Total Marks: 20

Course: Computer Vision & Robotics | Coverage: Lab 1 & Lab 2

Marks: Q1–Q7 (2 pts each) | Q8 (6 pts)

Student Name: _____ **Student ID:** _____ **Date:** _____

Q1. **[Medium MCQ]** A computer vision dataset contains color images with shape (450, 800, 3) and data type uint8. What are the image height, image width, and the exact uncompressed memory size occupied by a single image array in bytes?

A) Height = 800, Width = 450, Memory = 1,080,000 bytes
B) Height = 450, Width = 800, Memory = 1,080,000 bytes
C) Height = 450, Width = 800, Memory = 360,000 bytes
D) Height = 800, Width = 450, Memory = 360,000 bytes

Q2. **[Medium MCQ]** An engineer needs to adjust the brightness and contrast of a color image without causing unnatural color shifts or tint distortions. Which color space decouples lightness/luminance into an isolated channel independent of chromatic color channels?

A) RGB
B) BGR
C) Grayscale
D) LAB (or HSV)

Q3. **[Medium MCQ]** A scientific imaging lab needs to upscale satellite imagery by 8× for terrain inspection. They require the highest quality interpolation utilizing an 8×8 sinc-windowed neighborhood to eliminate jagged edge artifacts. Which OpenCV interpolation flag should they specify?

A) cv2.INTER_NEAREST
B) cv2.INTER_LINEAR
C) cv2.INTER_LANCZOS4
D) cv2.INTER_AREA

Q4. **[Medium MCQ]** A robot arm picks up square components from a conveyor belt that arrive rotated 90 degrees clockwise from their nominal orientation. Which OpenCV function call reorients the captured frame by exactly 90 degrees clockwise?

A) cv2.rotate(frame, cv2.ROTATE_90_CLOCKWISE)
B) cv2.rotate(frame, cv2.ROTATE_90_COUNTERCLOCKWISE)
C) cv2.rotate(frame, cv2.ROTATE_180)
D) cv2.flip(frame, 1)

Q5. **[Challenging MCQ]** Analyze the following Python snippet:
img = np.zeros((160, 160, 3), dtype=np.uint8); sub = img[40:120, 40:120]
sub[:] = [255, 255, 0]; print(img[80, 80].tolist())
What will be printed to the console?

A) [0, 0, 0]
B) [255, 255, 0]
C) [0, 255, 255]
D) [255, 255, 255]

Q6. **[Challenging MCQ]** Consider two single-pixel 8-bit unsigned integer arrays evaluated in Python:
p1 = np.array([220], dtype=np.uint8); p2 = np.array([60], dtype=np.uint8)
out1 = cv2.add(p1, p2)[0][0]; out2 = (p1 + p2)[0]
What are the values of out1 and out2?

A) out1 = 280, out2 = 280
B) out1 = 24, out2 = 255
C) out1 = 255, out2 = 255
D) out1 = 255, out2 = 24

Q7. **[Challenging MCQ]** An image img has dimensions (height=480, width=640). A vision system needs to rotate the image 30° counter-clockwise around its center at 1.0× scale. What are the correct arguments for cv2.getRotationMatrix2D and cv2.warpAffine?

A) M = cv2.getRotationMatrix2D((240, 320), 30, 1.0); res = cv2.warpAffine(img, M, (480, 640))
B) M = cv2.getRotationMatrix2D((320, 240), -30, 1.0); res = cv2.warpAffine(img, M, (640, 480))
C) M = cv2.getRotationMatrix2D((320, 240), 30, 1.0); res = cv2.warpAffine(img, M, (640, 480))
D) M = cv2.getRotationMatrix2D((240, 320), 30, 1.0); res = cv2.warpAffine(img, M, (640, 480))

Q8. **[Written Scenario-Based Problem — 6 Marks]** **Topic: Perspective Correction & Homography**

Problem Scenario: A document scanner robot captures an image of a certificate lying flat on a table, but because the camera looks from an oblique angle, the document appears distorted as an irregular quadrilateral (trapezoid) with non-parallel perspective converging lines. The four corner coordinates of the certificate in the photo are known (pts_src), and the robot needs to rectify it into a flat, top-down rectangular view of size 800×600 with destination coordinates pts_dst.

(a) Most Appropriate OpenCV Function: _____

(b) Essential Function Call / Parameters: _____

(c) Technical Justification & Suitability:

_____

_____

INSTRUCTOR COPY — DO NOT DISTRIBUTE TO STUDENTS

OpenCV Practical Lab Assessment — Complete Answer Key, Detailed Rationales & Rubrics

Master Quick-Grading Matrix (All Groups)

Group	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8 Expected OpenCV Function
Group One	B	D	A	C	A	B	D	cv2.copyMakeBorder()
Group Two	C	A	B	D	C	A	B	cv2.equalizeHist()
Group Three	A	C	D	B	A	C	B	cv2.addWeighted()
Group Four	D	B	A	C	D	B	A	cv2.bitwise_and()
Group Five	B	D	C	A	B	D	C	cv2.getPerspectiveTransform() and cv2.warpPerspective()

DETAILED REFERENCE & RUBRIC: GROUP ONE

Q#	Ans	Technical Rationale (Why Correct)	Key Distractor Analysis & Student Misconceptions
Q1	B	In NumPy/OpenCV, img.shape is indexed as (rows, columns, channels), meaning Height = 400 and Width = 600. Since each uint8 pixel value occupies 1 byte, total memory = 400 × 600 × 3 × 1 = 720,000 bytes.	A and D confuse rows (height) with columns (width). C forgets the 3 color channels (calculating 400 × 600 × 1).
Q2	D	OpenCV reads images natively in BGR channel order, whereas Matplotlib expects RGB. Converting from BGR to RGB via COLOR_BGR2RGB resolves the inverted red/blue channels.	A erroneously attempts an RGB-to-BGR conversion on an already BGR buffer. B converts to single-channel grayscale. C returns a reversed list of channels rather than a merged 3D image array.
Q3	A	cv2.INTER_AREA uses pixel area resampling (averaging pixel footprints), making it the optimal and recommended method for downsampling/shrinking to avoid severe aliasing and moiré artifacts.	INTER_NEAREST causes severe blocky decimation. INTER_LINEAR and INTER_CUBIC are designed for upscaling/zooming and introduce aliasing or excessive overhead when downsampling.
Q4	C	In OpenCV cv2.flip(src, flipCode), flipCode == 0 specifies flipping vertically around the horizontal x-axis.	A corresponds to flipCode == 1 (horizontal flip). D corresponds to flipCode == -1. B requires cv2.rotate().
Q5	A	NumPy basic slicing produces a shallow view that shares memory with the source array. Modifying roi[:] mutates the underlying memory of img at those coordinates.	B assumes NumPy slices are copies (they are views unless .copy() is called). C misinterprets color assignment. D is mathematically false as (100, 100) is well within the 200x200 array.
Q6	B	OpenCV's cv2.add() enforces saturation arithmetic (clamping sums exceeding 255 to 255). NumPy's + operator enforces 8-bit modular arithmetic: (210 + 70) % 256 = 280 % 256 = 24.	A assumes standard unbound integers. C reverses the behavior of OpenCV and NumPy. D assumes NumPy also clamps at 255.
Q7	D	OpenCV center coordinates require (x, y) = (width/2, height/2) = (300, 200). Positive angle 45 denotes counter-clockwise rotation. The output size dsize in warpAffine is (width, height) = (600, 400).	A, B, and C confuse (y, x) row-column indexing with (x, y) Cartesian coordinate parameters, or invert destination dimensions.

Q8 Written Scenario Solution & Grading Rubric (Group One):

• **Target Function:** cv2.copyMakeBorder() | **Sample Call:** padded = cv2.copyMakeBorder(src, top=80, bottom=80, left=0, right=0, borderType=cv2.BORDER_CONSTANT, value=[0, 0, 0])
• **Technical Explanation:** copyMakeBorder performs optimized border synthesis in C++ with zero manual memory re-allocation bugs. It cleanly supports various border replication/constant modes and preserves original pixel data intact.
• **Grading Scheme (6 Marks Total):** Function identification: 2 marks | Correct syntax and parameters (top/bottom=80, BORDER_CONSTANT, value): 2 marks | Valid technical explanation of aspect ratio / memory efficiency: 2 marks | Total: 6 marks.

DETAILED REFERENCE & RUBRIC: GROUP TWO

Q#	Ans	Technical Rationale (Why Correct)	Key Distractor Analysis & Student Misconceptions
Q1	C	For 2D grayscale arrays, img.shape returns (height, width) = (500, 800) with 1 channel. Total uncompressed memory = 500 × 800 × 1 × 1 byte = 400,000 bytes.	A inverts height and width. B assumes 3 channels for a grayscale image (calculating 500 × 800 × 3 = 1,200,000). D combines both errors.
Q2	A	Hue represents color shade/wavelength. Because degree angles [0, 360) exceed the 8-bit capacity of uint8, OpenCV scales Hue values by dividing by 2, mapping them strictly into [0, 179].	B describes Saturation. C describes Value. D states the standard mathematical range in degrees, which cannot fit into an 8-bit unsigned integer without overflow.
Q3	B	cv2.INTER_CUBIC utilizes bicubic interpolation over a 4×4 pixel neighborhood, generating smooth contours and high visual fidelity for upscaling.	INTER_NEAREST produces blocky pixelation. INTER_LINEAR uses a 2×2 neighborhood. INTER_AREA is tailored for downsampling.
Q4	D	In OpenCV cv2.flip(src, flipCode), flipCode == 1 (or any positive integer) designates a horizontal flip across the vertical y-axis.	A flips vertically. B flips along both axes. C rotates 180 degrees (which inverts both dimensions).
Q5	C	Because .copy() was explicitly invoked, roi points to a detached, independent memory buffer. Modifying roi[:] has zero effect on the original image img.	A overlooks the .copy() invocation. B and D propose fictitious side-effects.
Q6	A	OpenCV's cv2.subtract() uses saturation arithmetic, clamping underflow results to 0: max(0, 40 - 90) = 0. NumPy evaluates modular arithmetic: (40 - 90) % 256 = -50 % 256 = 206.	B reverses OpenCV and NumPy behaviors. C assumes signed integer representation. D incorrectly assumes NumPy also clamps to 0.
Q7	B	Center coordinates must be (x, y) = (width/2, height/2) = (250, 150). Counter-clockwise angle is +90, scale is 1.5. Output size dsize in warpAffine is (width, height) = (500, 300).	A swaps x and y center coordinates and destination dimensions. C applies a clockwise rotation (-90). D fails to apply the 1.5 scale factor.

Q8 Written Scenario Solution & Grading Rubric (Group Two):

• **Target Function:** cv2.equalizeHist() | **Sample Call:** enhanced = cv2.equalizeHist(gray_img)
• **Technical Explanation:** equalizeHist calculates the cumulative distribution function (CDF) of the image histogram and uses it as a monotonic remapping transfer function. Densely populated intensity bins are stretched widely across the [0, 255] spectrum, maximizing global contrast.
• **Grading Scheme (6 Marks Total):** Function identification: 2 marks | Correct data type/channel requirements (single-channel 8-bit uint8): 2 marks | Valid technical explanation of CDF / histogram stretching: 2 marks | Total: 6 marks.

DETAILED REFERENCE & RUBRIC: GROUP THREE

Q#	Ans	Technical Rationale (Why Correct)	Key Distractor Analysis & Student Misconceptions
Q1	A	NumPy shape is (rows, columns, channels) = (height, width, channels). Height = 300, Width = 700. Memory = 300 × 700 × 3 × 1 byte = 630,000 bytes.	B reverses height and width. C and D compute memory for a single channel (300 × 700 × 1).
Q2	C	OpenCV calculates grayscale using perceptual luminance weights based on human ocular sensitivity: Green ~0.587, Red ~0.299, and Blue ~0.114.	A is unweighted averaging which ignores human spectral sensitivity. B and D are non-standard heuristics that do not match ITU-R standards.
Q3	D	cv2.INTER_NEAREST simply assigns each destination pixel the value of the single closest source pixel without arithmetic filtering or multi-pixel averaging, making it the fastest algorithm.	INTER_LINEAR, INTER_CUBIC, and INTER_LANCZOS4 perform weighted interpolations across 2x2, 4x4, and 8x8 windows respectively, demanding substantially more CPU cycles.
Q4	B	Setting flipCode == -1 in cv2.flip() commands OpenCV to flip simultaneously across both the horizontal and vertical axes (equivalent to a 180° rotation).	A flips vertically only. C flips horizontally only. D rotates by 90 degrees.
Q5	A	In NumPy, slicing creates a memory view. Modifying elements of patch directly modifies the underlying memory of img. Pixel (50, 60) falls inside the sliced bounding box and is updated to [0, 0, 255].	B assumes slices are independent copies. C reverses the channel order. D is invalid as indices are valid.
Q6	C	cv2.add() enforces saturation arithmetic clamped to 255: min(255, 190 + 80) = 255. NumPy performs 8-bit unsigned integer wraparound: (190 + 80) % 256 = 270 % 256 = 14.	A assumes unbound integers. B swaps the two behaviors. D assumes NumPy also clamps.
Q7	B	A 2D translation matrix is defined as [[1, 0, tx], [0, 1, ty]] where tx = 60 and ty = 40. The destination size in warpAffine is (width, height) = (600, 400).	A swaps tx and ty and swaps output dimensions. C inverts the output tuple dimensions. D mutates identity diagonal coefficients.

Q8 Written Scenario Solution & Grading Rubric (Group Three):

• **Target Function:** cv2.addWeighted() | **Sample Call:** blended = cv2.addWeighted(feedA, 0.65, feedB, 0.35, 0.0)
• **Technical Explanation:** addWeighted implements the linear blend formula dst = alpha*src1 + beta*src2 + gamma using saturation arithmetic. Pixel values exceeding 255 are automatically clamped rather than wrapping around modulo 256, ensuring artifact-free blending.
• **Grading Scheme (6 Marks Total):** Function identification: 2 marks | Correct parameters and syntax (feedA, 0.65, feedB, 0.35, 0): 2 marks | Valid technical explanation of saturation arithmetic vs overflow: 2 marks | Total: 6 marks.

DETAILED REFERENCE & RUBRIC: GROUP FOUR

Q#	Ans	Technical Rationale (Why Correct)	Key Distractor Analysis & Student Misconceptions
Q1	D	In NumPy/OpenCV, img.shape indicates (height, width, channels) = (600, 900, 3). Total memory = 600 × 900 × 3 × 1 byte = 1,620,000 bytes.	A and C invert height and width. B computes memory for only 1 channel.
Q2	B	The b channel is a 2D intensity map where pixel values directly quantify the magnitude of blue light (0 = no blue, 255 = maximum blue intensity).	A is too restrictive (pure blue pixels also show as 255). C is the opposite of reality. D is false as split is lossless.
Q3	A	By default, OpenCV uses bilinear interpolation (cv2.INTER_LINEAR) when no explicit interpolation argument is supplied to cv2.resize().	INTER_NEAREST is faster but not default. INTER_CUBIC and INTER_AREA require explicit specification.
Q4	C	cv2.rotate() with flag cv2.ROTATE_90_COUNTERCLOCKWISE rotates an image exactly 90 degrees counter-clockwise.	A rotates 90 degrees clockwise. B performs a 180-degree flip. D rotates 180 degrees.
Q5	D	Because .copy() was called on the sliced subarray, crop is allocated in a separate memory buffer. Mutating crop[:] to 0 does not alter the original array img[75, 75].	A assumes crop was a view sharing memory. B and C propose non-existent mathematical modifications.
Q6	B	cv2.subtract() uses saturation arithmetic, clamping negative numbers to 0: max(0, 30 - 100) = 0. NumPy performs unsigned 8-bit modular arithmetic: (30 - 100) % 256 = -70 % 256 = 186.	A inverts the operations. C assumes signed integer representation. D assumes NumPy also clamps.
Q7	A	Leftward translation corresponds to tx = -50, downwards corresponds to ty = 80. Destination tuple dsize in warpAffine is (width, height) = (700, 500).	B swaps tx and ty. C reverses destination dimensions (height, width). D corrupts transformation identity coefficients.

Q8 Written Scenario Solution & Grading Rubric (Group Four):

• **Target Function:** cv2.bitwise_and() | **Sample Call:** masked_turf = cv2.bitwise_and(img, img, mask=mask)
• **Technical Explanation:** Bitwise AND performs logical conjunction bit-by-bit. Where mask=255 (binary 11111111), pixel values are preserved intact because x AND 255 = x. Where mask=0 (binary 00000000), output bits become 0 because x AND 0 = 0, resulting in pure black.
• **Grading Scheme (6 Marks Total):** Function identification: 2 marks | Correct syntax and parameters (img, img, mask=mask): 2 marks | Valid technical explanation of bitwise logic (x AND 255 = x, x AND 0 = 0): 2 marks | Total: 6 marks.

DETAILED REFERENCE & RUBRIC: GROUP FIVE

Q#	Ans	Technical Rationale (Why Correct)	Key Distractor Analysis & Student Misconceptions
Q1	B	NumPy shape is (rows, columns, channels) = (height, width, channels). Height = 450, Width = 800. Total memory = 450 × 800 × 3 × 1 byte = 1,080,000 bytes.	A inverts height and width. C and D compute memory for a single grayscale channel.
Q2	D	CIELAB (and HSV) cleanly isolate luminance/lightness (the L or V channel) from color chrominance (a/b or H/S channels). Modifying only the luminance channel preserves color tint fidelity.	In RGB and BGR, luminance is entangled across all three channels. Grayscale discards all chromatic color information entirely.
Q3	C	cv2.INTER_LANCZOS4 applies Lanczos interpolation using an 8×8 pixel sinc-based kernel, offering the highest edge preservation and visual fidelity for large upscaling factors.	INTER_NEAREST and INTER_LINEAR use 1x1 and 2x2 neighborhoods. INTER_AREA is tailored for downsampling.
Q4	A	cv2.rotate() with flag cv2.ROTATE_90_CLOCKWISE rotates the image buffer 90 degrees clockwise.	B rotates counter-clockwise. C rotates 180 degrees. D mirrors horizontally.
Q5	B	Slicing creates a memory view. Modifying elements of sub mutates the underlying memory buffer of img directly. Pixel (80, 80) is inside the slice and is set to [255, 255, 0].	A assumes slicing makes a copy. C inverts the color values. D assumes white saturation.
Q6	D	cv2.add() clamps saturated values to 255: min(255, 220 + 60) = 255. NumPy performs modular arithmetic: (220 + 60) % 256 = 280 % 256 = 24.	A assumes unconstrained integers. B inverts OpenCV and NumPy behaviors. C assumes NumPy also clamps.
Q7	C	Center coordinate in Cartesian (x, y) is (width/2, height/2) = (320, 240). Positive angle 30 denotes counter-clockwise rotation. Destination size dsize is (width, height) = (640, 480).	A swaps center (y, x) and output dimensions. B specifies a clockwise rotation (-30). D swaps x and y center coordinates.

Q8 Written Scenario Solution & Grading Rubric (Group Five):

• **Target Function:** cv2.getPerspectiveTransform() and cv2.warpPerspective() | **Sample Call:** M = cv2.getPerspectiveTransform(pts_src, pts_dst)
rectified = cv2.warpPerspective(img, M, (800, 600))
• **Technical Explanation:** Perspective transformation uses an 8-DOF (3×3) projective matrix that maps 4 non-collinear point pairs, allowing converging perspective lines to become parallel. Affine transformations only have 6-DOF (requiring 3 points) and strictly preserve parallelism, making them unable to correct keystone distortion.
• **Grading Scheme (6 Marks Total):** Function identification (both getPerspectiveTransform and warpPerspective): 2 marks | Point pairs requirement (4 points vs 3 for affine): 2 marks | Valid technical explanation of perspective/parallelism: 2 marks | Total: 6 marks.